

Lec 55 Sequential Modeling with RNNs and LSTM

Lec 55 Sequential Modeling with RNNs and LSTM

Lecture	55 / 63 + intro
Duration	38:40
Captions	English (auto-generated)
Words	5,301
Video	omPaHVk9hrM
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone, welcome back to the lecture series on fundamentals of generative AI and large language models. In my previous session, we have discussed about the basics of NLP, the text prep-processing, tokenization, word embeddings and other things. In my today's session, we will speak about the sequential modeling with RNNs and LSTMs. Okay, sequential modeling of text with RNNs and LSTMs. Okay. So the contents for today's session goes like y sequence models understanding the temporal dependencies recural neural networks which are the RNN RNN architecture and computation types of RNN limitations of them long short-term memory which we call as LSTMs LSTM gates and cell states okay so we will start with

[00:57] what is the reason or requirement ment to have the sequential models. So if we see most of our real world data are they have an inherent order inside them. Okay. So when I speak I am speaking one sentence. So the order of the words matter. If I place one word which is supposed to come at the end of the s sent sentence I I place it in the initial part of the sentence the meaning of the sentence will change. Okay. So there is an inherent order while we are either speaking or writing in text.

[01:40] Okay. So sentence, speeches, all of them have an inherent order which is very very important to reveal the information that the person wants to give. Okay?

For example, just look at this. There are two sentences. The dog bites man. Man bites dog. So in the second sentence this man instead of being at the end it has come here. The same words are there. Dog bites man. Man bites dog. Even though the same words are used in both the sentences the meaning is completely different. Okay.

[02:27] which shows or reinforces that the sequential or the ordering of the words is very very important in any sentence. Okay. Now in NLP in our last session we have done we know what are the steps that are required for the text to be processed by any model. Okay. Like for example I like I love AI that is the sentence that is the text input. Then we know about tokenization that is the sentence is divided into tokens like I love AI and then for each of these tokens there is an vector embedding. Till this one it is clear to us. Now how can we process these vectors while preserving their sequential order? I have broken it. I love AI and there are some vector embeddings for all these three tokens.

[03:25] How can I process this entire sequence of words so that the order of the words remain the same? It is not changed. Okay. And that is where our sequential modeling comes into picture. Okay. And there are various applications as I was telling it requires a sequential modeling like word prediction. If I want to predict the word at the end of a sentence, I need to understand the meaning of the previous words and for that the sequential modeling or the order of the words is very important which is achieved by sequential modeling. Okay. Then for sentence classification like I have a sentence I love pizza. So if I want it to have a sentiment analysis like positive sentiment or negative sentiment, I need to understand

[04:18] what is the information or the meaning of the sentence for which the ordering of the words or tokens is very important. So that is also another application of sequential modeling. Similarly, translation from English to German, English to French, English to any other language or any one language to other language, it requires the sequential modeling. Okay. So that is with this motivation scientists came out with various models like RNNs and LSTMs. Okay.

[04:52] Now in sequential modeling the thing that is very important for us to understand as we are telling that the order of the words is very very important that means the concept that is where our concept of temporal dependency comes into picture. Temporal dependency means the data is dependent at which instant of time it is going into the model. That is called as the temporal dependency. Okay. So the movie is not good. So if I have this sentence, the movie is not good. Now this good is a word.

[05:36] It is having not just before good which changes the meaning of the word. Okay. Meaning of the sentence. So the meaning of a word depends on its preceding words. So what is coming just before that one based on that one the meaning of good here is dependent. If the word not is not there the me the movie is good that's a positive sentiment. The movie is not good the sentiment of the sentence changes. So

your preceding words they are very important to understand what meaning this text wants to give. Okay. Now this is called as shortterm dependency.

[06:27] That means the meaning of the word good is dependent on its just preceding word. So there is a dependency but this is a very short-term dependency. good is there just be you know before that one there is not so the dependency is this type of dependency is called as short-term dependency okay let us take another example I grew up in France I speak fluent dash there is a fill in the gap that we need to do or the model needs to predict I grew up in France I speak fluent dash Okay. So there is some what prediction that need the model needs to do. What will be the correct answer? Most probable answer is French. Okay. So I grew up in France and I speak fluent French. Okay.

[07:24] Now if [clears throat] we see this French, if I put French here, this has a relation with France. So the language that I am fluent in speaking is related to the place where I was staying. So French and France they are related. But if you see there are 1 2 3 three words in between France and French and this type of dependency is called as long-term dependency and if I want the model to remember this one model to predict uh the word French correctly Then the model needs to remember the earlier information which was 1 2 3 words before French. So I grew up in France. I speak fluent French. The dependency of the terms or the words French and France is separated by three words and these kind of dependency is called

[08:46] as long-term dependency. Okay. So we got to know what is short-term dependency that means just the pre previous word. If there are an in between two words which are very much related in between that there are lot of words like 3 4 5 6 then it is called as long-term dependency. Okay. Now the question that comes is can a feed forward neural network work in this scenario? Now if we see in a feed forward neural network the information travels in one direction.

[09:22] So you have the inputs x_1 x_2 x_3 x_4 . These are the input input neurons. Uh these things are the input neurons. Then you have one hidden layer here and then you have the output layer. In some of your other models you will have multiple hidden layers. Okay. So the thing that is to be noticed here is there is no memory of the previous inputs. So there is no way where what is the previous input that memory is stored in feed feed forward neural network. So the memory information gets lost here and that is the reason feed forward neural networks do not work for sequential modeling.

[10:07] Okay. Then we come to the introduction of recurral neural networks or which we call as RNN. What are they? What is the core idea behind that one? So we know we are working we are talking about sequential models. RNN is the first one that we will be speaking about. Okay. So the core idea here in RNN where instead of discarding the previous information carry it forward by introducing hidden states. So you have a

set of words in a sentence and when you are processing them sequentially you don't discard the previous information but you carry it forward through something called as the hidden state.

[10:54] This hidden state is a new concept in recurrent neural network. Okay. So if we see you know in a visual way you have the input to the RNN then there is a working memory which stores the input or the stores the memory of the previous word and then you use that one to predict your output. Okay. So this is how RNN work and the hidden states act as the network's memory. So there is a hidden state in RNN and that works as the memory of the network.

[11:34] Okay. So we will see how these RNNs work and what is the math behind that one so on and so forth. Okay. So if we unfold this one we see these are the inputs at time instant t . This is this is time equal to t . This is time step $t + 1$ and this is the time step t minus one. So these three time steps I have taken. Okay. And x is the input to the model. Sorry. Okay. Now in between the input and the output there is something called there is something called as the hidden states.

[12:27] Here you are the weights associated or the weights between the input and the hidden state. So h_{t-1} is the hidden state at the time stamp t minus one. h_t is the hidden state at time time step t small t and h_{t+1} is the hidden state at time $t + 1$. Okay. U is the weight matrix. And where is this u working? It is between your input and the hidden state. Okay. And this is there for all the time steps. Now when you are predicting it there is something else that is also happening. So this arrow means you are getting some input from your previous hidden state.

[13:27] Okay. V is the weight associated with these hidden states. And then from these hidden states we predict the output y_{t-1} $\hat{y}_t$ $\hat{y}_{t+1}$ okay which are the predicted words for t minus one time instant t th time instant $t + 1$ time instant. Okay. So if we see U is the weight matrices between input and hidden, V is the weight matrices associated with the pre previous hidden state and the current hidden state. Okay, these are called as the recurrent weights and W is the weight matrix associated between the hidden and the output layer. Okay. So that is the way and here what is this x_{t-1} x_t and x_{t+1} ? This is the sequence of words. I love machine learning. Okay. Or I love pizza. So I will come at x_{t-1} . Love will

[14:39] come in the next time step. Then pizza. Okay. So this is the sequence of words. That is where your temporal dependency comes into picture. Okay. And here as we have seen there is a hidden layer which keeps the information you know to and it uses it for the prediction. Okay. Now understanding the RNN little bit more with the math part. I have kept the figure here once again for our own ease of understanding. While we are trying to understand the math there are two things that is there. One is the hidden state update.

[15:23] At each instant of time, the hidden states gets updated. Okay. So at every

instance of time, the hidden states get updated. Updated with what? Updated with the previous information. Okay. And it is once it is updated then there is another thing that is there which is the output calculation or the prediction. Okay. So you give an input, it will go through the hidden state. The hidden states at each and every time step it will get updated with the input as well as the information from the previous hidden states. So we will see at each time steps t the RNN updates its hidden state using the current input x_t . So if I consider this hidden state h_t okay it will get updated with the current input x_t and the previous hidden

[16:27] state h_{t-1} . What is there in h_{t-1} ? It is the information that was there in my time step $t-1$. Okay. So that is used for getting the updated h_t . Okay. So the equation is like this $h_t = \tanh(v h_{t-1} + u x_t + b_h)$ where v is the weights associated with your previous hidden states and the current hidden state. Okay. So this is also v . This is also so this is the u weights and that multiplies with h_{t-1} for getting h_t then u is the weight here x_t is the or u is the weight here x_t is the input here at that time step t is the bias term okay so b_h is the bias term associated with the hidden state okay so this is the equation

[17:35] for updating the hidden state. Okay. So that is the first thing that at each and every time step the hidden state gets updated because that is required one every time step we are going forward in the sequence and if we are not able to update our hidden state that means we are not getting the information from the just preceding word. Okay. So that is the reason the hidden state updates are required. Now come to the output calculation. What is this output here?

[18:12] That is $\hat{y}_t$ which is the predicted output. Why? And it is calculated using the hidden state h_t which in turn depends on x_t . We have seen h_t depends on x_t and this h_t is used for predicting $\hat{y}_t$. Okay. So, $\hat{y}_t = g(W h_t + b_y)$ where g is an activation function. It depends on what type of application we are using. For multiclass classification, it will be soft max and if it is for binary classification, it will be sigmoid activation function. So depending on the application, the activation function is chosen. And then W into h_t . W into h_t means what is W ? W is the weight between the hidden and the output. Output predicted and b_y is the bias terms. Okay. So this is the core equations for RNNs.

[19:23] Okay. And the same weight matrices U , V , and W are shared across all time steps. Now we look into the various types of RNN. Okay. The first type is one to one. What do I mean? You have an input sequence of one word and then there is the RNN block and then you have an output which again is one word. Okay. So this means the input this is the RNN block and this is the output. Okay. Now what is the application?

[20:02] Where do we use this kind of architecture in binary classification? Suppose

I give an image of a cat and I want to say whether it is a cat or dog. It will tell cat. So there is no you know sequential dependency. One to one it is okay. Then one to many. In one to many what happens? There is one input and these are the RNN blocks and you have many output 1 2 3. So this is one to many type of RNA. What is the application? Image captioning. Suppose your input is an image.

[20:47] The image is about the sunset in a riverside. So the image is one input. Now when you are telling your model to do a captioning for the image, it will require few words. Sunset near sea or sunset near the river. So there are multiple words. Okay. So that those places we have one to many. So here I have put three but it can go from go to any number. Okay. depending on the number of words that are there in your output caption for that particular image.

[21:30] Then we have many to one where we have many input words and then these are the RNN blocks and then you have one output and what is the application for this one that is sentiment analysis. So in sentiment analysis say I love pizza. So I love pizza. This is my input. What is the output? I want either positive or negative. So this is a positive sentiment more than this one I don't want. Right? Positive sentiment. So this is many to one. Okay. And then can be there can be many to many where you have many number of inputs and then these are the RNN blocks and you have many outputs. Okay.

[22:16] where it is used for language translation, for sequence labeling or parts of speech tagging. Okay, these are various NLP tasks. Okay, I play cricket. So if I write it like this, I play cricket and I want a parts of speech tagging. I is basically a pronoun, play is a verb and cricket is a noun. So these three are required. So how many number of words you have in the sequence those many number of outputs you require in the parts of speech tagging. Okay.

[23:04] So these are the various types of RNN. Now we have to see is there any limitation of RNN. Just take the example the my brother lives in France and I visit him every summer which is why I understand French quite well. Okay. As once again we see that there is a relationship between France and French but the number of words between them is 1 2 3 4 5 6 7 8 9 10 11 11 words. So the connection between the location and the language that he is speaking they are divide they are separated by 11 words. Okay. And what happens here is as we were telling that it goes through the hidden states by the time we predict French it has to go through so many hidden states H1, H2, H3, H4 till H11 sequence of hidden states and in this

[24:09] process the information starts gradually weakening. So when the time comes when it has to predict French the memory by that time might already weakened for the word France because there are so many updations of the hidden state that keeps happening. Okay. So RNN cannot handle long-term dependency. So this is a long-term dependency. Correct. So that is a limitation of RNN. And why is that one? It

is because of vanishing gradient problem. So as we are seeing there are so many sequence so many words in between and we know that in any neural network there is a back propagation in sequence back propagation of error in sequence sequential models it is back propagation through time every time step

[25:05] it is back propagating okay and through when it is back propagating through so so many number of time steps What happens is if the gradient value is small, multiplying a small number with another small number makes it smaller. Smaller number with another smaller number makes it even even more smaller. So at the end of this long chain of your hidden state transition, what happens is your gradient starts vanishing or becomes very very low. Okay. So this vanishing gradient problem is a major problem in uh you know or a major limitation in RNN and there can be exploding gradient also if the gradient is high or near to one.

[25:56] What happens is it will be multiplied while back propagating through time and then it will explode which will lead to instability in training. Okay. So exploding gradient and vanishing gradient these are the limitations because that because of which RNNs cannot handle long-term dependencies. Okay, what is the solution to that one? That is where our long short-term memory or LSTMs come into picture. So we'll just have a look what is this long short-term memory or LSTM. So if you have the LSTM block here we have the input then you have the working memory which is same as your normal RNN or vanilla RNN together with that one there is a long-term memory also so there is a immediate memory and the long-term memory okay so

[26:48] that is the difference and from there using both of these memory it will predict the output okay so in LSTMs it has two states one is the hidden state which was already there in your normal cardinace okay which is which will keep the memory of the preceding word and together with that one there is something called as the cell state it carries important information across multiple steps multiple time steps okay so that is the difference between RNN and LSTM in LSTMs there are gates that are used for transmission of the information so there are three gates that are used here for get gate input gate an output gate.

[27:32] These gates are also neural networks which are learnable functions and each gate learns how much information to be retained, updated or exposed. Okay, so these gates are actually controlling how much information they should they should keep, how much information they should pass it forward, how much information they should forget. Okay. So we'll have a look into the LSTM architecture. So we have three blocks here. And what are my these three blocks? This is time step t .

[28:09] This is time step t minus one. This is my time step $t + one$. This is my sequence t minus one. $T + one$. Okay. This is an LSTM chain with three nodes. Three nodes means t minus one, t and $t + 1$ time steps. Now we will see how the entire architecture works. I have taken in time step t . I am demonstrating the gates the

activity of the gates. There are three gates. The forget gate, the input gate and the output gate. These are three gates that are there. Now from the t minus 1 it state the state cell state information comes as c_{t-1} . So that is the state input from t minus one it state time stamp which goes to the forget gate and there is another input which is the hidden state which is h_{t-1} . So

[29:12] two states are coming one is the cell state and another one is the hidden state. And this is coming from t minus 1 time stamp. So we have written c_{t-1} and h_{t-1} . Now we will see what happens in the forget gate. In the forget gate if you see there is a concatenation of x_t which is the current input vector at time stamp t . Okay. So we'll have a look into what exactly happens in the forget gate. So this is the forget gate. a zoomed view of what the operations that are happening inside the forget gate. So we have this h_{t-1} and x_t coming inside the forget gate. After that there is a sigmoid activation function that is working on this concatenation of h_{t-1} and x_t and we get out of that one f_t .

[30:00] What is my f_t ? f_t equal to sigmoid of $W_f \cdot [h_{t-1} || x_t] + b_f$. Okay. Now [clears throat] so the concatenation comes from here. Okay. And then we get f_t and then W_f is the weight matrix associated with the forget gate. Now this f_t the value of f_t lies between 0 and 1. It can either completely forget or it can completely remember. Okay. So that is the uh that is the work of the f_t . Okay. It will tell how much to forget or how much to remember. Then there was another one cell state c_{t-1} which we have already seen that it has come

[30:50] inside the forget gate and there is some operation that happens between f_t and c_{t-1} which is basically your element wise multiplication and we get something as $f_t \odot c_{t-1}$ element wise multiplication with c_{t-1} . So this f_t actually determines the f_t value lies between zero and one. If it is very near to one that means it has almost retained all the information. If it is very near to zero it has almost forgotten all the information. Okay. So what happens here is this is what is coming out of my forget gate. Okay. f_t element wise multiplication with c_{t-1} .

[31:35] Then we will see what happens. So this is the operation that is happening inside your forget gate. Then we will look into what is happening inside my input gate. So this is my input gate. And if you see once again this um concatenation of x_t and h_{t-1} goes inside and there is once more the sigmoid activation function which works on the concatenation of h_{t-1} and x_t and then something comes out of that one which is your i_t . So what is this i_t ? i_t is equal to sigmoid of $W_i \cdot [h_{t-1} || x_t] + b_i$ i_t is the bias for the input gate associated with the input gate. Okay. So this is what I get here. And then there is

one more function one more operation that is

[32:28] happening that is your $\tanh$ uh activation function is happening on the concatenation of h_t minus one and x_t and then we get $\hat{c}_t$ which we call as the candidate sales state and $\hat{c}_t$ equal to $\tanh(w_{hc} h_t + 1 x_t + b_c)$ b_c is the bias term associated with the uh you know this uh candidate sales state. Okay, candidate sales state tells what new information we need to keep there based on our current uh input. Okay, then what is this W_i and W_c ? W_i is the weight matrix associated with the input gate weight and W_c is the weight matrix associated with the candidate cell.

[33:16] Okay, so $\hat{c}_t$ is the candidate cell uh state. Okay. And then there is once again an element wise multiplication between it and $\hat{c}_t$. And we get something here like this. And this it into $\hat{c}_t$ element wise multiplication. And then this whatever output is coming here it goes and gets added here with the pre the input that is coming from your this um forget gate. So what we are getting here is f_t element wise multiplication $\hat{c}_t$ minus one this is coming from here and this term is coming from here. Okay.

[33:57] So this is an addition there is a addition term there. Then we will look at what happens in the output gates. So we will just write it here. This is the output gate operation that is happening here operations. So once again the this is the concatenation between h_t minus 1 x_t which goes to the sigmoid activation function and we get o_t what is this o_t is given by sigmoid w_o into $h_t + 1 x_t + b_o$ b_o is the bias term associated with the output gate and w_o what is this w_o is the weight matrix associated with the output gate okay and then there is another operation it is The $\tanh$ which is happening with $\hat{c}_t$ $\tanh$ is getting operated on $\hat{c}_t$ and then whatever output we are getting that is getting element

[34:54] wise multiplic multiplic multiplied with O_t . So what is the output that I'm getting after all these operations of $\tanh$ working on $\hat{c}_t$ and then element wise operation is my H_t . So let's have a look into what is this H_t . H_t is nothing but your O_t element wise multiplication of $\tanh H$ of $\hat{c}_t$. So $\tanh H$ of $\hat{c}_t$ is coming from here. Okay. So this is my H_t . Now where is this H_t going now? If you see my previous one, this is where it is going.

[35:29] H_t is going to the $T + 1$ state. Okay. And similarly my $\hat{c}_t$ that is coming out here, it is going here. So $\hat{c}_t$ and H_t are going to the $T + 1$ it state and the same operations will happen there also. So this is about the LSTM architecture how its operations are happening inside that one. Okay. And with that one I come to the end of today's discussion. Before I wrap up let us quickly have a look into what we have studied today. So we have start started with what is the requirement of sequential modeling. We require sequential models because there is an inherent order that is present in sentences or speeches when we speak.

[36:14] Okay. And it is very important to have a meaningful information exchange

and that was not possible with feed forward neural network because there is no memory that is associated. So we started with RNN which has a hidden state which keeps the memory of the just the previous word. But then we saw what is after that we saw the mathematical equations for that one and but the uh what was the limitation of this one was RNN were not able to capture long-term dependency. So when there is a relationship between two words which are separated by say 10 to 12 words in between them RNN were not able to keep keep the you know information intact because of vanishing gradient and exploding gradient exploding gradient

[37:02] problem. Okay. Now to solve that one LSTMs came into picture and LSTMs what is the difference between RNN and LSTMs? LSTMs have another state together with the hidden state that is the cell state and that cell state is basically used for keeping you know past information. Okay. Much much better than the it is not only keeping the previous word information but long whatever has passed by in a lot of uh or a couple of uh time steps time steps that information is stored in your cell state and then we have seen what are the operations that are happening in the LSTM architecture.

[37:44] There are the gates. How are the gates working? The forget gate, the input gate and the output gate and how the entire LSTM architecture works. Okay. So with this one I come to the conclusion of today's session of mine. In the next session we will look into the evolution of LSTMs to transformers because after LSTMs there are limitations of these LSTMs also when the uh dependency is too too long. So that is where istium started facing challenge and then came into picture the transformers which works mainly on attention mechanisms.

[38:22] Okay. So that will be the topic of discussion for my next session. Uh till we meet for the next session. Uh please continue reading and happy learning. Thank you all.